Comparing GPT and Rosette for Program Synthesis

Ern Kim, Jineshwar Nariani, Lana Vu, Manovay Sharma, Mishti Gala, Nikhila Peravali
Mentor: Angela Upreti

Motivating Questions

- Program Synthesis Tools: How well can tools automatically generate code?
- State-of-the-Art Comparison: What are the best tools available and how do they stack up against each other?
- Computers vs. Programmers: Can computers replace human programmers?

Background

- Traditional programming involves manual coding, but recent advances in Language Models (LMs) and program synthesis tools offer automation potential.
- Oue research is a small-scale exploration of LLMs' ability to generate code automatically, raising questions about the efficacy of these tools compared to human programmers.
- We compare ChatGPT and Rosette's performance across a few programming related-tasks. Tasks we compare include bug detection, synthesis, optimization and verification. **Rosette** is a solver-aided programming language that extends **Racket.**

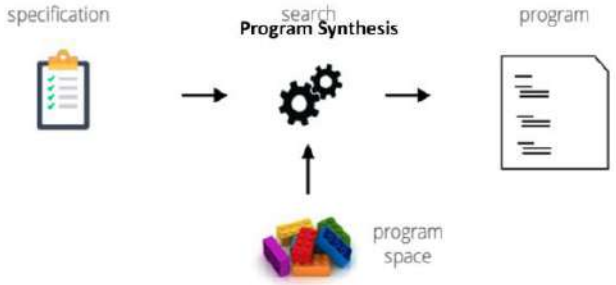


Image from UCSD Program Synthesis course by Nadia Polikarpova[2]

Program synthesis is a problem of finding a program P that satisfies the specification φ on all inputs.[3]

- **Specification:** φ may be a formula, a reference implementation, input / output pairs, traces, demonstrations, text-based description like in GPT etc.[3] Rosette uses **formula** as specification and GPT uses **text description** as specification.
- **Search:** Search strategy can be enumerative, constraint-based solver-aided search, stochastic search. Rosette uses an SMT **solver** to find a correct program. GPT uses NLP algorithms to predict code.
- **Program space:** GPT allows limiting program space by language. We limited program search to Racket language. For Rosette, we made use of grammar based program specification for synthesis tasks.

Methods

ChatGPT

Rosette

GenProg
Evolutionary Program Repair

We assessed 3 tools: GenProg, Sketch, and Rosette, ultimately opting for Rosette due to its versatility in tasks it can perform, superior documentation, and thus, ease of use.

We tasked both ChatGPT and Rosette with solving 20 diverse code generation problems, covering scenarios from identifying problematic input values for functions to generating intricate program sections from specifications.

Example Synthesis Task: Check if an integer fits in BV bits

```
#lang rosette/safe

(require rosette/lib/synthax)

;; Very tiny bitvectors
(define BV 8)
(define bvI? (bitvector BV))


(define (int-prop-iff impl x)
  (<=>  (impl x) (equal? x (bitvector->integer (integer->bitvector  x (bitvector BV))))))


(define-symbolic x integer?)

; Grammar for synthesis
(define-grammar (int-gm x)
  [expr
   (choose x (?? integer?)
           ((bop) (expr) (expr)))]
  [bop
   (choose <= <)])

(define (intI-gm_?? x)
  (and (int-gm x #:depth 2)
       (int-gm x #:depth 2)))

(define solution
  (synthesize
   #:forall (list x)
   #:guarantee (assert (int-prop-iff intI-gm_?? x))))

(if (sat? solution) (print-forms solution) (print "UNSAT"))


(define (can-be-represented-with-8-bits-gpt? x)
  (and (integer? x)
       (<= -128 x 127)))

(define (sBVbits x)
  (assume (can-be-represented-with-8-bits-gpt? x))
  (assert (equal? x (bitvector->integer (integer->bitvector x (bitvector BV))))))


(verify (sBVbits x))
```

Declaration of a symbolic constant of integer type

Grammar that the synthesized program should abides by

Program to be synthesized. #:depth 2 allows for depth of two in the expression tree.

Synthesis Query

Output:(define (intI-gm_?? x) (and (< -129 x) (< x 128)))

Program synthesized by GPT

Verification Query to check if GPT synthesized a correct program

Output: (unsat)

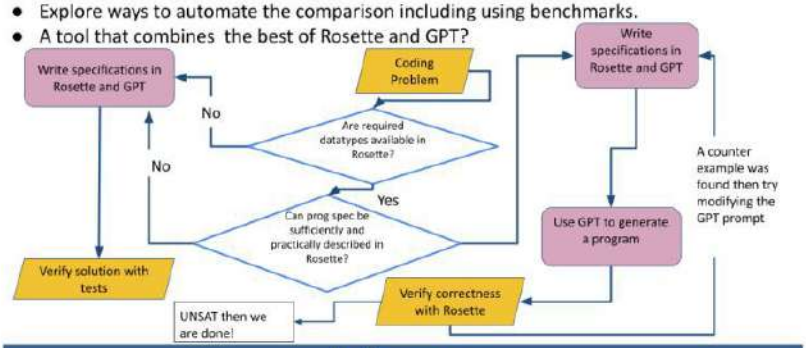
GPT Prompt: Write a racket program that takes in an integer x and returns true if signed integer x can be represented with 8 bits.

Findings

Correctness	Rosette is better for correctness. Unlike GPT, if the specification is both correct and complete, Rosette guarantees the generation of a correct program.
Specification	Rosette needs specifications and often times, complete specifications can be difficult to write. GPT's specifications are simpler to write, and even vague ones can result in program generation. However, this comes with a trade-off in terms of correctness.
Program Space	Rosette works on a small subset of datatypes it deems safe and is restricted to languages supported by Rosette. In contrast, GPT is capable of generating programs across a wide array of languages. However, the output is often incorrect. Rosette is suitableif we want to define and generate programs for a Domain Specific Language.
Speed	GPT is much much better than Rosette in terms of time for synthesis, and bug finding. Rosette is faster at verification than synthesis.

Future Work

- While we considered only 20 programs for our initial exploration, large scale experiments based on our initial findings/ hypothesis would be interesting.
- Explore ways to automate the comparison including using benchmarks.
- A tool that combines the best of Rosette and GPT?



References

1. Torlak, E., & Bodik, R. (2013). Growing solver-aided languages with rosette.
2. Polikarpova, N. (2020,Aug). *Program Synthesis*[PowerPoint slides].UCSD.
3. Torlak, E. (2019, Nov). *Program Synthesis*[PowerPoint slides].CSE 507 Fall 19, University of Washington.
4. Solar-Lezama, A. (2008). *Program synthesis by sketching* [Phd].
5. Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). GenProg: A Generic Method for Automatic Software Repair.